1.Campus Event Financial Planning

1. Problem Statement

Campus events such as technical fests, cultural programs, workshops, and seminars require proper financial planning. The organizers must manage expenses related to venue booking, food, decoration, and other miscellaneous activities within the available budget.

The objective of this case study is to develop a Python program that calculates the total expenses of a campus event and determines whether the event is within the allocated budget or exceeds it.

The program accepts the total budget and individual expense amounts as input, calculates the total expenditure, and displays the remaining budget or the amount by which the budget is exceeded.

2. Objectives

- To understand the use of variables and input/output statements in Python.
- To perform arithmetic operations using Python.
- To calculate the total expenses of a campus event.
- To compare total expenses with the allocated budget.
- To use conditional statements to determine the budget status.
- To develop a simple financial planning application.

3. Assumptions

1. The total budget is entered by the user.
2. All expense amounts are entered as positive numerical values or zero.
3. The expenses include venue, food, decoration, and other miscellaneous costs.
4. All amounts are expressed in the same currency.

5. The program assumes that the user enters valid numerical inputs.
6. The event is considered within budget when total expenses are less than or equal to the allocated budget.

4. Algorithm

The following algorithm describes the step-by-step procedure for solving the problem.

Algorithm: Campus Event Financial Planning

1. Start.
2. Read the total budget.
3. Read the venue cost.
4. Read the food cost.
5. Read the decoration cost.
6. Read the other expenses.
7. Calculate the total expenses using:

$$\text{Total} = \text{Venue} + \text{Food} + \text{Decoration} + \text{Other}$$

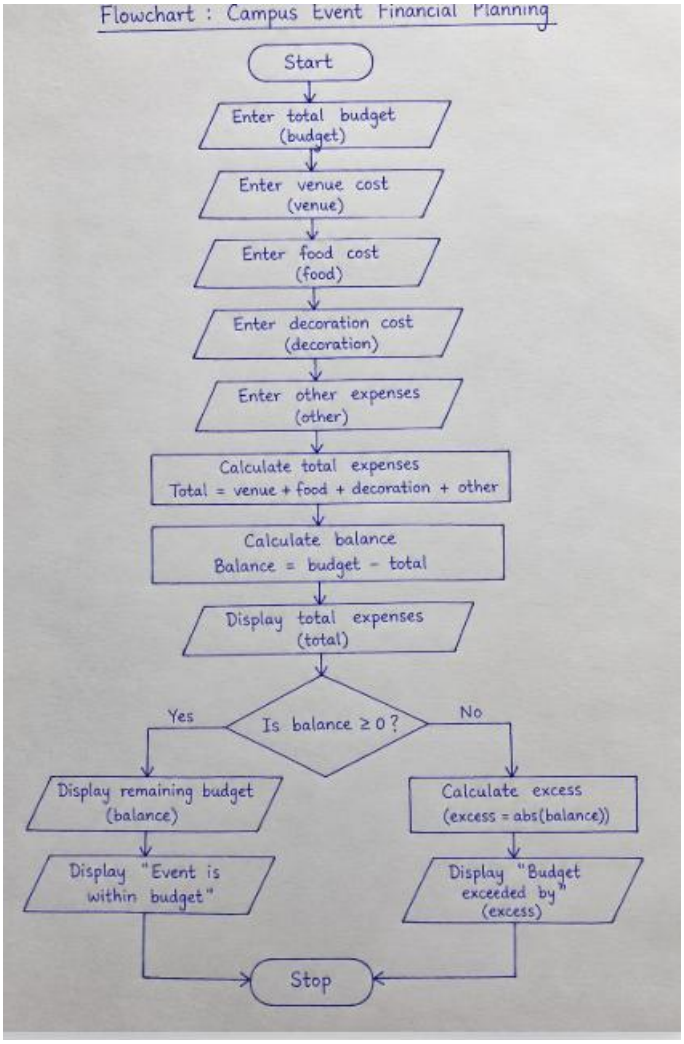
8. Calculate the balance using:

$$\text{Balance} = \text{Budget} - \text{Total}$$

9. Display the total expenses.
10. Check whether the balance is greater than or equal to zero.
11. If the condition is true:
    - Display the remaining budget.
    - Display "Event is within budget".
12. Otherwise:
    - Calculate the excess amount using the absolute value of the balance.
    - Display "Budget exceeded by" the excess amount.

13. Stop.

5. Flowchart



6. Pseudocode

BEGIN

INPUT budget

INPUT venue

INPUT food

INPUT decoration

INPUT other

```
    total = venue + food + decoration + other

    balance = budget - total

    PRINT "Total Expenses:", total

    IF balance >= 0 THEN

        PRINT "Remaining Budget:", balance

        PRINT "Event is within budget"

    ELSE

        PRINT "Budget exceeded by:", ABS(balance)

    END IF

END
```

7. Python Source Code

The following Python program uses only basic Python concepts such as input statements, variables, arithmetic operators, conditional statements, and output statements.

```
# Campus Event Financial Planning
```

```
budget = float(input("Enter the total budget: "))
venue = float(input("Enter venue cost: "))
food = float(input("Enter food cost: "))
decoration = float(input("Enter decoration cost: "))
other = float(input("Enter other expenses: "))

total = venue + food + decoration + other
balance = budget - total

print("Total Expenses:", total)

if balance >= 0:
    print("Remaining Budget:", balance)
    print("Event is within budget")
else:
    print("Budget exceeded by:", abs(balance))
```

Explanation of the Python Code

Statement	Explanation
float(input())	Accepts numerical input, including decimal values.
total = venue + food + decoration + other	Calculates the total expenses.
balance = budget - total	Calculates the remaining budget.
print()	Displays the result.
if balance >= 0	Checks whether the event is within budget.

else	Executes when the expenses exceed the budget.
abs(balance)	Converts the negative balance into a positive excess amount.

8. Test Cases

The following test cases are used to verify the correctness of the program.

Test Case	Budget	Venue	Food	Decoratio	Other	Expected Result
TC1	50000	10000	15000	5000	3000	Within budget
TC2	30000	10000	12000	5000	5000	Budget exceeded
TC3	40000	10000	15000	10000	5000	Within budget
TC4	20000	5000	5000	5000	5000	Within budget

Test Case Calculations

TC1:

Total expenses:

10000+15000+5000+3000=33000

Remaining budget:

50000-33000=17000

Result: Event is within budget.

TC2:

Total expenses:

10000+12000+5000+5000=32000

Budget exceeded:

32000-30000=2000

Result: Budget exceeded by 2000.

TC3:

Total expenses:

10000+15000+10000+5000=40000

Remaining budget:

40000-40000=0

Result: Event is within budget.

TC4:

Total expenses:

5000+5000+5000+5000=20000

Remaining budget:

20000-20000=0

Result: Event is within budget.

Test Case Calculations

TC1:

Total expenses:

10000+15000+5000+3000=33000

Remaining budget:

50000-33000=17000

Result: Event is within budget.

TC2:

Total expenses:

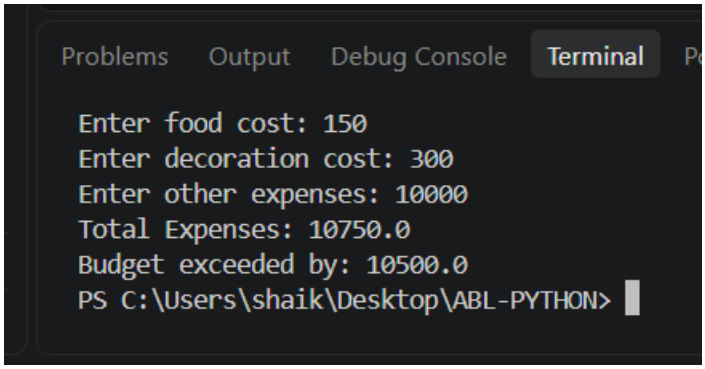
10000+12000+5000+5000=32000

Budget exceeded:

32000-30000=2000

Result: Budget exceeded by 2000

9.Output



10. Conclusion

The Campus Event Financial Planning program was successfully developed using Python programming concepts.

The program accepts the total budget and individual expenses, calculates the total expenditure, and compares it with the allocated budget. Based on the comparison, it displays the remaining budget or the amount by which the budget is exceeded.

This case study helped in understanding variables, input/output operations, arithmetic operators, conditional statements, and the use of the abs() function in Python.

The program provides a simple and effective solution for managing campus event expenses and supports better financial planning.

2.Examination Result and Grace-Mark Validation

1. Problem Statement

In educational institutions, students are evaluated based on their examination marks. A student must obtain a minimum of 35 marks out of 100 to pass the examination. However, students who score between 30 and 34 marks may be eligible for grace marks to help them pass.

The objective of this case study is to develop a Python program that validates the marks entered by the user and determines the examination result.

The program accepts marks out of 100. If the marks are below 0 or above 100, it displays "Invalid marks". If the marks are 35 or above, the student is declared as Pass. If the marks are between 30 and 34, the program adds 5 grace marks and declares the student as Pass. Otherwise, the student is declared as Fail.

2. Objectives

- To understand the use of input and output statements in Python.
- To validate examination marks using conditional statements.
- To implement nested if-else statements.
- To determine whether a student has passed or failed.
- To apply grace marks based on the examination rules.
- To develop a simple examination result validation system.

3. Assumptions

1. The examination is conducted for 100 marks.
2. The passing mark is 35.
3. Students scoring between 30 and 34 are eligible for 5 grace marks.
4. Students scoring below 30 are declared Fail.
5. Students scoring 35 or above are declared Pass.

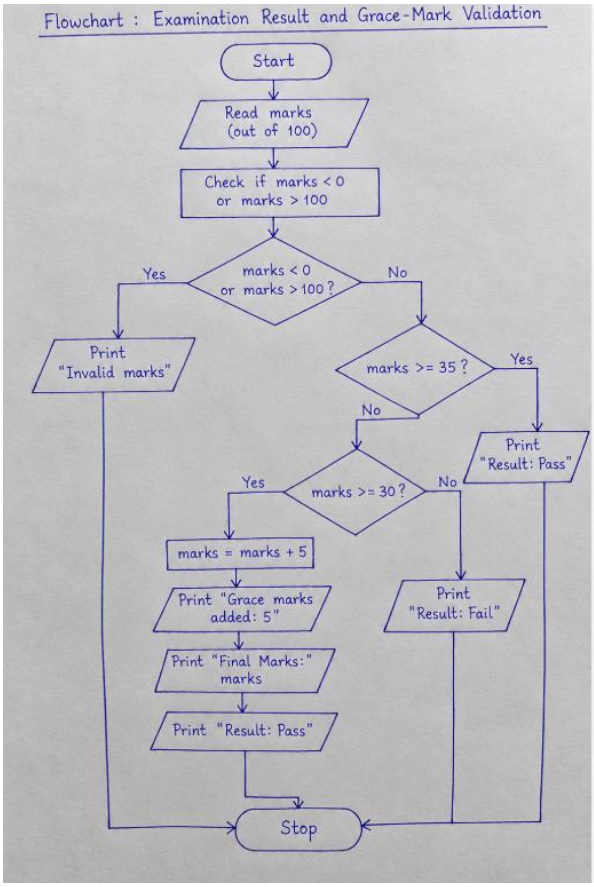
6. Marks below 0 or above 100 are considered invalid.
7. The program accepts integer marks only.
8. Grace marks are added only when the marks are between 30 and 34.

4. Algorithm

Algorithm: Examination Result and Grace-Mark Validation

1. Start.
2. Read the examination marks out of 100.
3. Check whether the marks are less than 0 or greater than 100.
4. If the marks are invalid:
    - Display "Invalid marks".
5. Otherwise, check whether the marks are greater than or equal to 35.
6. If the condition is true:
    - Display "Result: Pass".
7. Otherwise, check whether the marks are greater than or equal to 30.
8. If the condition is true:
    - Add 5 grace marks to the original marks.
    - Display "Grace marks added: 5".
    - Display the final marks.
    - Display "Result: Pass".
9. Otherwise:
    - Display "Result: Fail".
10. Stop.

5. Flowchart



6.Pseudocode

BEGIN

INPUT marks

IF marks < 0 OR marks > 100 THEN

PRINT "Invalid marks"

ELSE

IF marks >= 35 THEN

```
        PRINT "Result: Pass"

    ELSE IF marks >= 30 THEN

        marks = marks + 5

        PRINT "Grace marks added: 5"

        PRINT "Final Marks:", marks

        PRINT "Result: Pass"

    ELSE

        PRINT "Result: Fail"

    END IF

  END IF

END
```

7.Python Source code

```
# Examination Result and Grace-Mark Validation

marks = int(input("Enter marks out of 100: "))
```

```
if marks < 0 or marks > 100:
   print("Invalid marks")
else:
   if marks >= 35:
      print("Result: Pass")
   elif marks >= 30:
      marks = marks + 5
      print("Grace marks added: 5")
      print("Final Marks:", marks)
      print("Result: Pass")
   else:
      print("Result: Fail")
```

8. Test Cases

The following test cases are used to verify the correctness of the program.

Test Case	Input Marks	Expected Output	Condition
TC1	-5	Invalid marks	Marks below 0
TC2	101	Invalid marks	Marks above 100
TC3	75	Result: Pass	Marks above 35
TC4	35	Result: Pass	Exact passing marks

Test Case Calculations

TC1: Invalid Marks

Input:

Marks=-5

Since marks are below 0, the result is:

Invalid marks

TC2: Invalid Marks

Input:

Marks=101

Since marks are above 100, the result is:

Invalid marks

TC3: Direct Pass

Input:

Marks=75

Since 75 is greater than or equal to 35:

Result: Pass

TC4: Exact Passing Marks

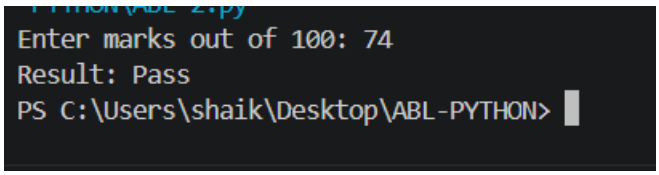
Input:

Marks=75

Since 35 is the passing mark:

Result: Pass

9.Output



10.Conclusion

The Examination Result and Grace-Mark Validation program was successfully developed using Python programming concepts.

The program validates the marks entered by the user and determines whether the student has passed or failed the examination. It also applies 5 grace marks to eligible students who score between 30 and 34 marks.

This case study helped in understanding input/output statements, comparison operators, logical operators, nested conditional statements, and decision-making in Python.

The program provides a simple and effective method for examination result validation and grace-mark calculation.

3.Attendance Recovery Computation

1. Problem Statement

Attendance is an important requirement for students in educational institutions. Students are generally required to maintain a minimum attendance percentage of 75% to be eligible for examinations.

The objective of this case study is to develop a Python program that calculates the attendance percentage of a student and determines whether the attendance requirement is satisfied.

The program accepts the total number of classes held and the number of classes attended. It validates the attendance details, calculates the attendance percentage, and checks whether it is at least 75%.

If the attendance percentage is below 75%, the program calculates the minimum number of additional classes the student must attend continuously without missing any class to achieve the required attendance percentage.

2. Objectives

- To understand input and output operations in Python.
- To calculate attendance percentage using arithmetic operators.
- To validate attendance details using conditional statements.
- To implement if-else statements for decision-making.
- To understand the use of while loops.
- To calculate the minimum additional classes required to achieve 75% attendance.
- To develop a simple attendance recovery computation system.

3. Assumptions

1. The total number of classes held must be greater than zero.
2. The number of classes attended must be greater than or equal to zero.

3. The number of classes attended cannot exceed the total classes held.
4. The minimum required attendance percentage is 75%.
5. All inputs are entered as integers.
6. Additional classes are assumed to be attended continuously without missing any class.
7. The program calculates the minimum number of additional classes required to satisfy the attendance requirement.
8. If the attendance is already 75% or above, no additional classes are required.

4. Algorithm

Algorithm: Attendance Recovery Computation

1. Start.
2. Read the total number of classes held.
3. Read the number of classes attended.
4. Set the target attendance percentage to 75.
5. Check whether the attendance details are valid:
    - If total classes held is less than or equal to zero, or
    - Attended classes are less than zero, or
    - Attended classes are greater than total classes held.
6. If the details are invalid, display "Invalid attendance details".
7. Otherwise, calculate the attendance percentage using:

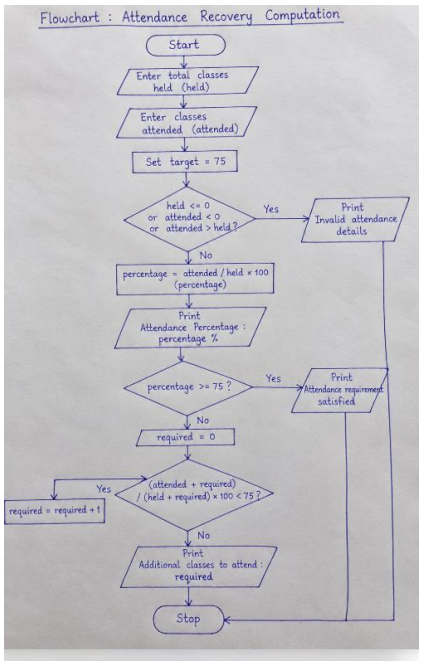
$$\text{Percentage} = \frac{\text{Attended}}{\text{Held}} \times 100$$

8. Display the attendance percentage.
9. Check whether the percentage is greater than or equal to 75.
10. If the condition is true:

- Display "Attendance requirement satisfied".

11. Otherwise:
    - Initialize required = 0.
    - Check whether the attendance percentage after attending additional classes is still below 75%.
    - If it is below 75%, increment required by 1.
    - Repeat until the attendance percentage reaches 75%.
    - Display the number of additional classes required.
12. Stop.

5. Flowchart



6.Pseudocode

BEGIN

INPUT held

INPUT attended

```
target = 75

IF held <= 0 OR attended < 0 OR attended > held THEN

   PRINT "Invalid attendance details"

ELSE

   percentage = attended / held * 100

   PRINT "Attendance Percentage:", ROUND(percentage, 2)

   IF percentage >= target THEN

      PRINT "Attendance requirement satisfied"

   ELSE

      required = 0

      WHILE (attended + required) /
          (held + required) * 100 < target DO

         required = required + 1
```

```
        END WHILE

        PRINT "Additional classes to attend:", required

    END IF

  END IF

END
```

7.Python Source code

```
held = int(input("Enter total classes held: "))
attended = int(input("Enter classes attended: "))
target = 75

if held <= 0 or attended < 0 or attended > held:
  print("Invalid attendance details")
else:
  percentage = attended / held * 100
  print("Attendance Percentage:", round(percentage, 2), "%")

  if percentage >= target:
    print("Attendance requirement satisfied")
  else:
    required = 0
    while (attended + required) / (held + required) * 100 < target:
```

```
    required += 1


print("Additional classes to attend:", required)
```

8. Test Cases

The following test cases are used to verify the correctness of the program.

Test Case	Classes Held	Classes Attended	Expected Result
TC1	100	80	Attendance requirement satisfied
TC2	100	75	Attendance requirement satisfied
TC3	100	60	Additional classes to attend: 60

Test Case Calculations

TC1: Attendance Requirement Satisfied

Input:

- Classes held = 100
- Classes attended = 80

Attendance percentage:

$$\frac{80}{100} \times 100 = 80\%$$

Since 80% is greater than 75%, the attendance requirement is satisfied.

TC2: Exact 75% Attendance

Input:

- Classes held = 100
- Classes attended = 75

$$\frac{75}{100} \times 100 = 75\%$$

Result: Attendance requirement satisfied.

TC3: Attendance Recovery

Input:

- Classes held = 100
- Classes attended = 60

Current attendance:

Let \(x\) be the additional classes required.

$$\frac{60}{100} \times 100 = 60\%$$

Let \(x\) be the additional classes required.

$$\frac{60 + x}{100 + x} \times 100 \geq 75$$

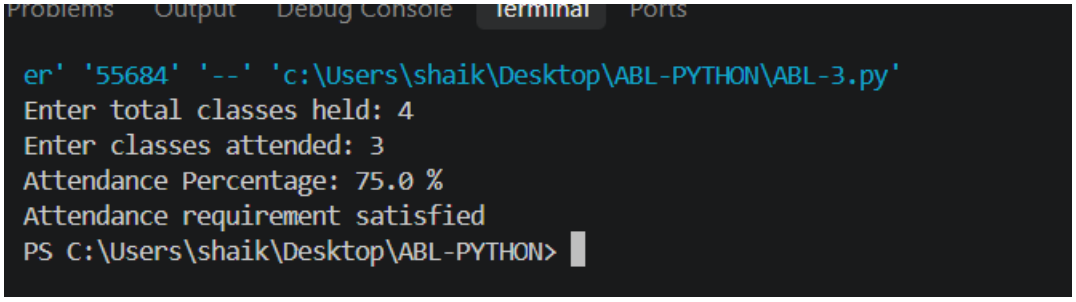
$$60 + x \geq 75 + 0.75x$$

$$0.25x \geq 15$$

$$x \geq 60$$

Therefore, the student must attend 60 additional classes continuously.

9.Output



10. Conclusion

The Attendance Recovery Computation program was successfully developed using Python programming concepts.

The program calculates the attendance percentage of a student and checks whether the minimum attendance requirement of 75% is satisfied. If the attendance is below the required percentage, it calculates the minimum number of additional classes the student must attend continuously to achieve the target.

This case study helped in understanding variables, arithmetic operators, conditional statements, logical operators, and while loops in Python.

The program provides a simple and effective solution for attendance monitoring and recovery computation.

4. ATM Transaction Sequence Simulation

1. Problem Statement

Automated Teller Machines (ATMs) provide users with convenient access to banking services such as balance enquiry, cash deposits, and cash withdrawals.

The objective of this case study is to develop a Python program that simulates the basic operations of an ATM system.

The program accepts the initial account balance and displays a menu containing four options: Check Balance, Deposit, Withdraw, and Exit.

The user can select an operation repeatedly until the Exit option is selected. The program validates deposit and withdrawal amounts, updates the account balance accordingly, and displays suitable messages for invalid amounts and insufficient balance.

2. Objectives

- To understand input and output operations in Python.
- To implement menu-driven programming.
- To use while loops for repeated transactions.
- To implement conditional statements using if-elif-else.
- To perform arithmetic operations on account balances.
- To validate deposit and withdrawal amounts.
- To simulate basic ATM transaction operations.

3. Assumptions

1. The initial account balance is entered by the user.
2. The initial balance is assumed to be non-negative.
3. The deposit amount must be greater than zero.
4. The withdrawal amount must be greater than zero.

5. A withdrawal is allowed only when the amount is less than or equal to the current balance.
6. The user can perform multiple transactions in one session.
7. The program terminates when the user selects option 4.
8. All monetary values are entered as numerical values.

4. Algorithm

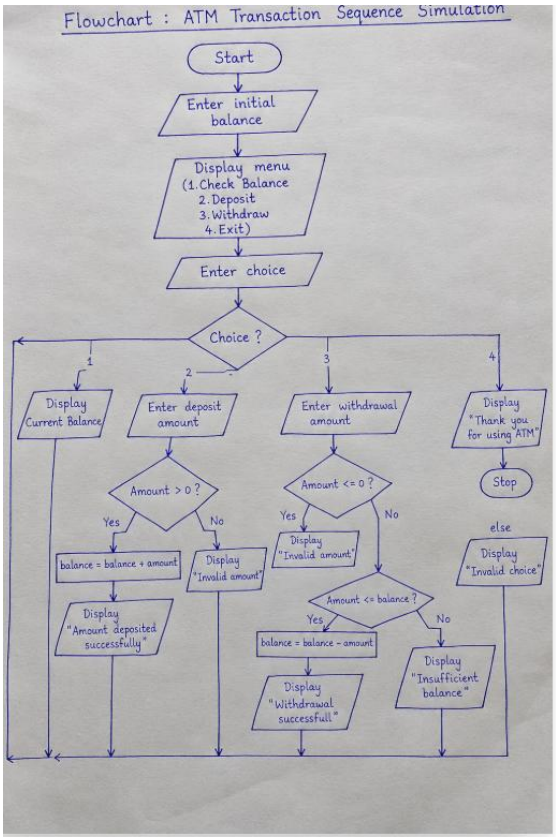
Algorithm: ATM Transaction Sequence Simulation

1. Start.
2. Read the initial account balance.
3. Display the ATM menu:
    - Check Balance
    - Deposit
    - Withdraw
    - Exit
4. Read the user's choice.
5. If the choice is 1:
    - Display the current balance.
6. If the choice is 2:
    - Read the deposit amount.
    - If the amount is greater than zero, add it to the balance.
    - Display "Amount deposited successfully".
    - Otherwise, display "Invalid amount".
7. If the choice is 3:
    - Read the withdrawal amount.
    - If the amount is less than or equal to zero, display "Invalid amount".

- Otherwise, if the amount is less than or equal to the balance, subtract it from the balance.
- Display "Withdrawal successful".
- Otherwise, display "Insufficient balance".

8. If the choice is 4:
   - Display "Thank you for using ATM".
   - Terminate the loop.
9. If the choice is invalid:
   - Display "Invalid choice".
10. Repeat the menu until the user selects Exit.
11. Stop.

5. Flowchart



6. Pseudocode

BEGIN

```
INPUT balance

WHILE TRUE DO

  PRINT "1. Check Balance"
  PRINT "2. Deposit"
  PRINT "3. Withdraw"
  PRINT "4. Exit"

  INPUT choice

  IF choice = 1 THEN

    PRINT "Current Balance:", balance

  ELSE IF choice = 2 THEN

    INPUT amount

    IF amount > 0 THEN

      balance = balance + amount

      PRINT "Amount deposited successfully"
```

```
    ELSE
        PRINT "Invalid amount"
    END IF
ELSE IF choice = 3 THEN
    INPUT amount
    IF amount <= 0 THEN
        PRINT "Invalid amount"
    ELSE IF amount <= balance THEN
        balance = balance - amount
        PRINT "Withdrawal successful"
    ELSE
        PRINT "Insufficient balance"
```

```
        END IF

    ELSE IF choice = 4 THEN

        PRINT "Thank you for using ATM"

        BREAK

    ELSE

        PRINT "Invalid choice"

    END IF

  END WHILE

END
```

7.Python Source Code

```
balance = float(input("Enter initial balance: "))

while True:
  print("\n1. Check Balance")
  print("2. Deposit")
  print("3. Withdraw")
  print("4. Exit")
```

```
choice = int(input("Enter your choice: "))

if choice == 1:
    print("Current Balance:", balance)

elif choice == 2:
    amount = float(input("Enter deposit amount: "))
    if amount > 0:
        balance += amount
        print("Amount deposited successfully")
    else:
        print("Invalid amount")

elif choice == 3:
    amount = float(input("Enter withdrawal amount: "))
    if amount <= 0:
        print("Invalid amount")
    elif amount <= balance:
        balance -= amount
        print("Withdrawal successful")
    else:
        print("Insufficient balance")

elif choice == 4:
```

```
        print("Thank you for using ATM")
        break

    else:
        print("Invalid choice")
```

8. Test Cases

Test Case	Initial Balance	Choice	Amount	Expected Result
TC1	5000	1	—	Displays current balance
TC2	5000	2	2000	Deposit successful
TC3	5000	2	-500	Invalid amount

Test Case Calculations

TC1: Check Balance

Initial balance = 5000

The user selects option 1.

Result:

Current Balance: 5000.0

TC2: Deposit

Initial balance = 5000

Deposit amount = 2000

New balance:

5000+2000=7000

Result: Amount deposited successfully.

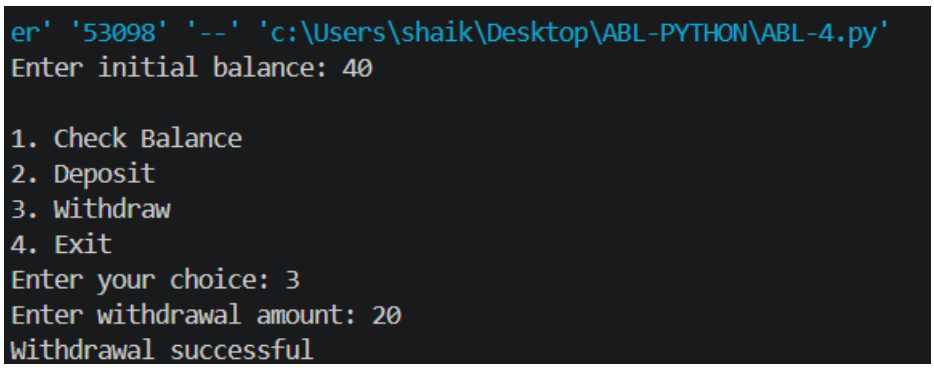
TC3: Invalid Deposit

Deposit amount = -500

Since the amount is not greater than zero:

Result: Invalid amount.

9.Output



10. Conclusion

The ATM Transaction Sequence Simulation program was successfully developed using Python programming concepts.

The program simulates basic ATM operations such as checking the account balance, depositing money, withdrawing money, and exiting the system. It validates transaction amounts and checks for sufficient balance before allowing withdrawals.

This case study helped in understanding variables, arithmetic operators, conditional statements, while loops, and the break statement in Python.

The program provides a simple and effective simulation of ATM transactions and demonstrates the use of menu-driven programming.

5. Fuel Consumption and Route Comparison

1. Problem Statement

Fuel consumption and travel costs are important factors when selecting a route for a journey. Different routes may have different distances, which can affect the amount of fuel consumed and the total travel cost.

The objective of this case study is to develop a Python program that compares the fuel consumption and fuel cost of two routes.

The program accepts the distances of Route 1 and Route 2, the vehicle's mileage, and the fuel price per litre. It validates the inputs, calculates the fuel required and the total fuel cost for each route, and identifies the cheaper route.

If both routes have the same fuel cost, the program displays that both routes have equal fuel cost.

2. Objectives

- To understand input and output operations in Python.
- To perform arithmetic calculations using Python.
- To calculate fuel consumption based on distance and mileage.
- To calculate the fuel cost for two different routes.
- To validate user inputs using conditional statements.
- To compare two routes using if-elif-else statements.
- To develop a simple fuel cost comparison system.

3. Assumptions

1. The distances of both routes are entered in kilometres.
2. Vehicle mileage is entered in kilometres per litre.
3. Fuel price is entered as the cost per litre.
4. Both route distances must be greater than zero.
5. Vehicle mileage must be greater than zero.

6. Fuel price must be greater than or equal to zero.
7. The vehicle's mileage is assumed to be the same for both routes.
8. The comparison is based only on fuel cost, not travel time, traffic, or road conditions.

4. Algorithm

Algorithm: Fuel Consumption and Route Comparison

1. Start.
2. Read the distance of Route 1.
3. Read the distance of Route 2.
4. Read the vehicle mileage.
5. Read the fuel price per litre.
6. Check whether any input is invalid:
    - If either distance is less than or equal to zero, or
    - Mileage is less than or equal to zero, or
    - Fuel price is less than zero.
7. If the input is invalid, display "Invalid input"
8. Otherwise, calculate the fuel required for Route 1:

$$\text{Fuel1} = \frac{\text{Distance1}}{\text{Mileage}}$$

9. Calculate the fuel required for Route 2:

$$\text{Fuel2} = \frac{\text{Distance2}}{\text{Mileage}}$$

10. Calculate the fuel cost for both routes:
11.

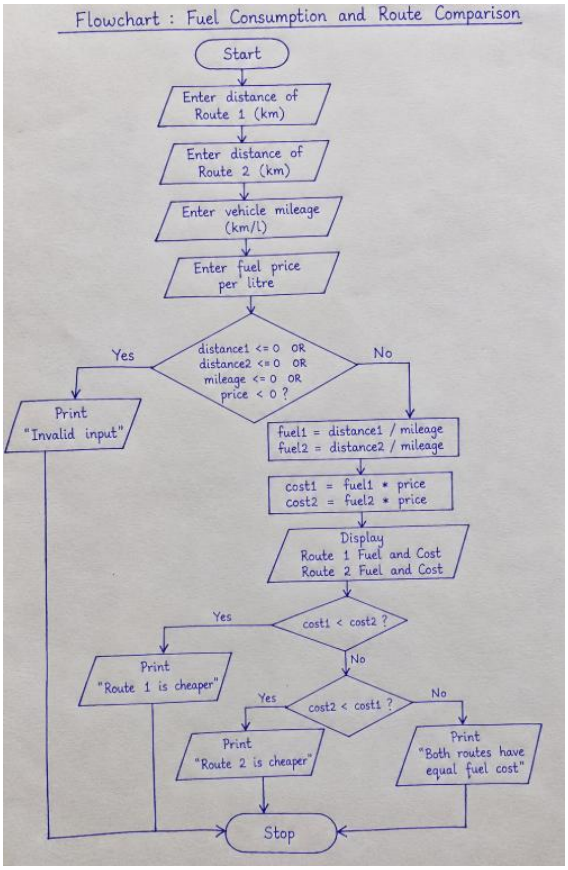
$$\text{Cost1} = \text{Fuel1} \times \text{Price}$$

12. Display the fuel consumption and cost of both routes.
13. Compare the costs:
    - If Cost1 is less than Cost2, display "Route 1 is cheaper".

- If Cost2 is less than Cost1, display "Route 2 is cheaper".
- Otherwise, display "Both routes have equal fuel cost".

14. Stop.

5. Flowchart



6. Pseudocode

BEGIN

INPUT distance1

INPUT distance2

INPUT mileage

INPUT price

IF distance1 <= 0 OR distance2 <= 0 OR

```
  mileage <= 0 OR price < 0 THEN

   PRINT "Invalid input"

ELSE

   fuel1 = distance1 / mileage
   fuel2 = distance2 / mileage

   cost1 = fuel1 * price
   cost2 = fuel2 * price

   PRINT "Route 1 Fuel:", ROUND(fuel1, 2)
   PRINT "Route 1 Cost:", ROUND(cost1, 2)

   PRINT "Route 2 Fuel:", ROUND(fuel2, 2)
   PRINT "Route 2 Cost:", ROUND(cost2, 2)

   IF cost1 < cost2 THEN

      PRINT "Route 1 is cheaper"

   ELSE IF cost2 < cost1 THEN

      PRINT "Route 2 is cheaper"
```

```
    ELSE

      PRINT "Both routes have equal fuel cost"

    END IF

  END IF

END
```

7.**Python Source Code**

```
    distance1 = float(input("Enter distance of Route 1 (km): "))
distance2 = float(input("Enter distance of Route 2 (km): "))
mileage = float(input("Enter vehicle mileage (km/l): "))
price = float(input("Enter fuel price per litre: "))

if distance1 <= 0 or distance2 <= 0 or mileage <= 0 or price < 0:
  print("Invalid input")
else:
  fuel1 = distance1 / mileage
  fuel2 = distance2 / mileage

  cost1 = fuel1 * price
  cost2 = fuel2 * price
```

```
print("Route 1 Fuel:", round(fuel1, 2), "litres")
print("Route 1 Cost:", round(cost1, 2))

print("Route 2 Fuel:", round(fuel2, 2), "litres")
print("Route 2 Cost:", round(cost2, 2))

if cost1 < cost2:
    print("Route 1 is cheaper")
elif cost2 < cost1:
    print("Route 2 is cheaper")
else:
    print("Both routes have equal fuel cost")
```

8. Test Cases

The following test cases are used to verify the correctness of the program.

Test Case	Route 1 (km)	Route 2 (km)	Mileage (km/l)	Fuel Price	Expected Result
TC1	100	200	20	100	Route 1 is cheaper
TC2	200	100	20	100	Route 2 is cheaper
TC3	100	100	20	100	Both routes have equal fuel cost

Test Case Calculations

TC1: Route 1 is Cheaper

Input:

- Route 1 distance = 100 km

- Route 2 distance = 200 km
- Mileage = 20 km/l
- Fuel price = 100 per litre

Fuel required for Route 1:

$$\frac{100}{20} = 5 \text{ litres}$$

Fuel cost of Route 1:

5x100=500

Fuel required for Route 2:

$$\frac{200}{20} = 10 \text{ litres}$$

Fuel cost of Route 2:

10x100=1000

Result: Route 1 is cheaper.

TC2: Route 2 is Cheaper

Route 1 distance = 200 km

Route 2 distance = 100 km

Fuel cost of Route 1 = 1000

Fuel cost of Route 2 = 500

Result: Route 2 is cheaper.

TC3: Equal Fuel Cost

Both routes have a distance of 100 km.

Fuel required:

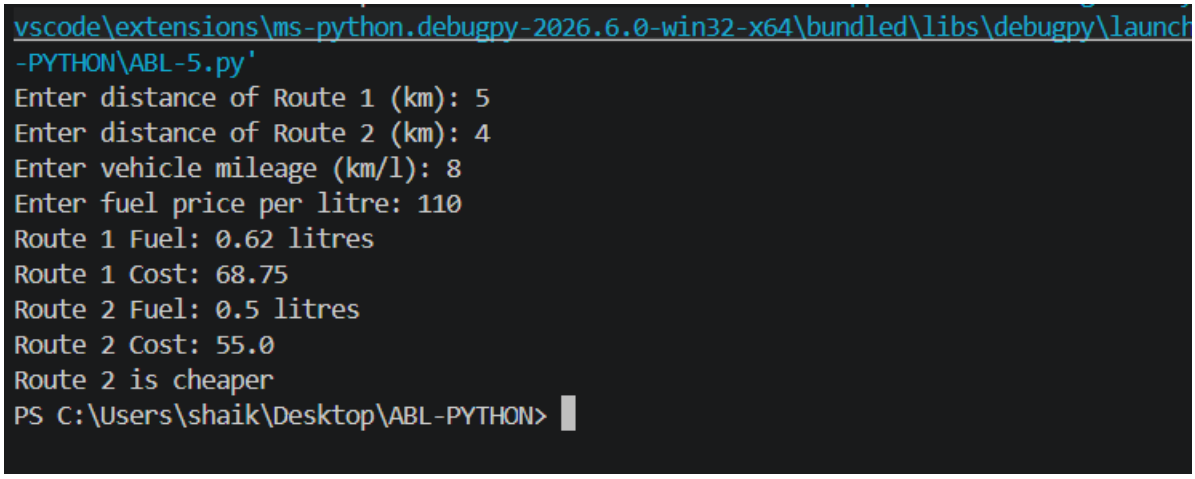
$$\frac{100}{20} = 5 \text{ litres}$$

Fuel cost:

5x100=500

Result: Both routes have equal fuel cost.

9.Output



10. Conclusion

The Fuel Consumption and Route Comparison program was successfully developed using Python programming concepts.

The program calculates the fuel consumption and fuel cost of two different routes based on their distances, vehicle mileage, and fuel price. It validates the inputs and compares the costs to identify the cheaper route.

This case study helped in understanding variables, input/output statements, arithmetic operators, conditional statements, and input validation in Python.

The program provides a simple and effective solution for comparing travel fuel expenses and supports better route selection based on fuel cost.

6. Multi-Product Shopping Budget Constraint

1. Problem Statement

In shopping, customers need to manage their expenses according to their available budget. When purchasing multiple products, it is important to calculate the total cost and determine whether the shopping expenses are within the budget.

The objective of this case study is to develop a Python program that calculates the total shopping cost for multiple products and checks whether the total expense exceeds the available budget.

The program accepts the shopping budget and the number of products. For each product, it accepts the price and quantity, calculates the total cost, and adds it to the overall shopping expense.

Finally, the program compares the total shopping cost with the budget. If the total cost is within the budget, it displays the remaining money. Otherwise, it displays the amount by which the budget is exceeded.

2. Objectives

- To understand input and output operations in Python.
- To calculate the total cost of multiple products.
- To use for loops for repeated product inputs.
- To implement conditional statements for budget validation.
- To perform arithmetic operations using price and quantity.
- To identify whether shopping expenses are within the budget.
- To calculate the remaining money or budget excess.

3. Assumptions

1. The shopping budget must be greater than or equal to zero.
2. The number of products must be greater than zero.
3. Product prices must be greater than or equal to zero.

4. Product quantities must be greater than zero.
5. The price of each product is entered per unit.
6. The total shopping cost is calculated by multiplying the price by the quantity of each product.
7. The program assumes that all entered products are purchased.
8. If any product details are invalid, the program stops processing further products.
9. The budget comparison is based only on the total product cost.

4. Algorithm

Algorithm: Multi-Product Shopping Budget Constraint

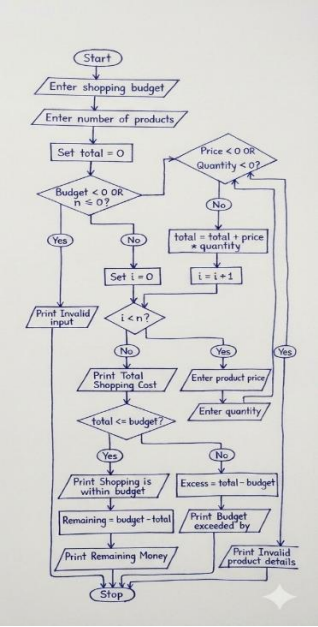
1. Start.
2. Read the shopping budget.
3. Read the number of products.
4. Initialize total = 0.
5. Check whether the budget is less than zero or the number of products is less than or equal to zero.
6. If the input is invalid, display "Invalid input".
7. Otherwise, repeat the following steps for each product:
    - Read the product price.
    - Read the product quantity.
    - Check whether the price is less than zero or the quantity is less than or equal to zero.
    - If the product details are invalid, display "Invalid product details" and stop processing.
    - Otherwise, calculate the product cost:

$$ \text{Product Cost}=\text{Price}\times\text{Quantity} $$

- Add the product cost to the total shopping cost.

8. If all products are valid, display the total shopping cost.
9. Compare the total shopping cost with the budget.
10. If the total cost is less than or equal to the budget:
    - Display "Shopping is within budget".
    - Calculate and display the remaining money.
11. Otherwise:
    - Calculate and display the amount by which the budget is exceeded.
12. Stop.

5.Flow Chart



6.Pseudo code

BEGIN

```
INPUT budget
INPUT n

total = 0

IF budget < 0 OR n <= 0 THEN

   PRINT "Invalid input"

ELSE

   FOR i = 0 TO n - 1 DO

      INPUT price
      INPUT quantity

      IF price < 0 OR quantity <= 0 THEN

         PRINT "Invalid product details"

         BREAK

      END IF
```

```
        total = total + price * quantity

    END FOR

    IF all products are valid THEN

        PRINT "Total Shopping Cost:", ROUND(total, 2)

        IF total <= budget THEN

            PRINT "Shopping is within budget"

            PRINT "Remaining Money:",
                ROUND(budget - total, 2)

        ELSE

            PRINT "Budget exceeded by:",
                ROUND(total - budget, 2)

        END IF

    END IF

END IF
```

END

7.Python Source code

```
budget = float(input("Enter your shopping budget: "))
n = int(input("Enter number of products: "))

total = 0

if budget < 0 or n <= 0:
   print("Invalid input")
else:
   for i in range(n):
      price = float(input("Enter product price: "))
      quantity = int(input("Enter quantity: "))

      if price < 0 or quantity <= 0:
         print("Invalid product details")
         break

      total += price * quantity
   else:
      print("Total Shopping Cost:", round(total, 2))

      if total <= budget:
         print("Shopping is within budget")
```

```
        print("Remaining Money:", round(budget - total, 2))
    else:
        print("Budget exceeded by:", round(total - budget, 2))
```

8. Test Cases

The following test cases are used to verify the correctness of the program.

Test Case	Budget	Products	Product Details	Expected Result
TC1	2000	2	500 × 2, 300 × 1	Within budget
TC2	1000	2	600 × 1, 500 × 1	Budget exceeded by 100
TC3	1500	3	200 × 2, 300 × 1, 400 × 1	Within budget

TC1: Shopping Within Budget

Budget = 2000

Product 1:

500x2=1000

Product 2:

300x1=300

Total shopping cost:

1000+300=1300

Remaining money:

2000-1300=700

Result: Shopping is within budget.

TC2: Budget Exceeded

Budget = 1000

Product 1:

600x1=600

Product 2:

500x1=500

Total shopping cost:

600+500=1100

Budget exceeded by:

1100-1000=100

Result: Budget exceeded by 100.

TC3: Multiple Products

Budget = 1500

Total shopping cost:

$$(200 \times 2) + (300 \times 1) + (400 \times 1)$$

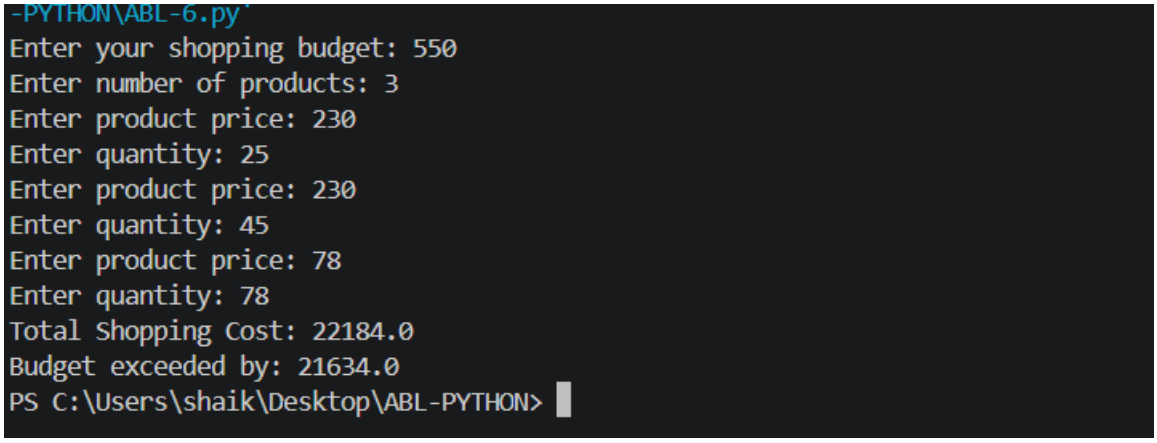
=400+300+400=1100

Remaining money:

1500-1100=400

Result: Shopping is within budget.

9.Output Screenshot



10. Conclusion

The Multi-Product Shopping Budget Constraint program was successfully developed using Python programming concepts.

The program calculates the total cost of multiple products based on their prices and quantities. It compares the total shopping cost with the available budget and determines whether the shopping expenses are within the budget or exceed it.

This case study helped in understanding variables, arithmetic operators, conditional statements, for loops, break statements, and for-else constructs in Python.

The program provides a simple and effective solution for managing shopping expenses and checking budget constraints.

7. Traffic Violation Penalty Calculator

1. Problem Statement

Traffic rules are essential for maintaining road safety and preventing accidents. Violations such as riding without a helmet, overspeeding, not wearing a seat belt, and signal violations may result in penalties.

The objective of this case study is to develop a Python program that calculates traffic penalties based on the type of violation.

The program displays a menu containing four traffic violations. The user selects a violation type, and the program assigns the corresponding fine amount.

If the user enters an invalid choice, the program displays "Invalid violation type". Otherwise, it displays the traffic penalty amount in rupees.

2. Objectives

- To understand input and output operations in Python.
- To implement menu-driven programming.
- To use if-elif-else statements for decision-making.
- To assign penalty amounts based on traffic violations.
- To validate user input.
- To develop a simple traffic violation penalty calculator.

3. Assumptions

1. The program considers four traffic violations.
2. No Helmet carries a penalty of Rs. 500.
3. Overspeeding carries a penalty of Rs. 1000.
4. No Seat Belt carries a penalty of Rs. 500.
5. Signal Violation carries a penalty of Rs. 1000.
6. The user enters an integer as the violation type.
7. Only choices from 1 to 4 are considered valid.
8. Any other choice is treated as an invalid violation type.
9. The penalty amounts are based on the values specified in the given Python program.

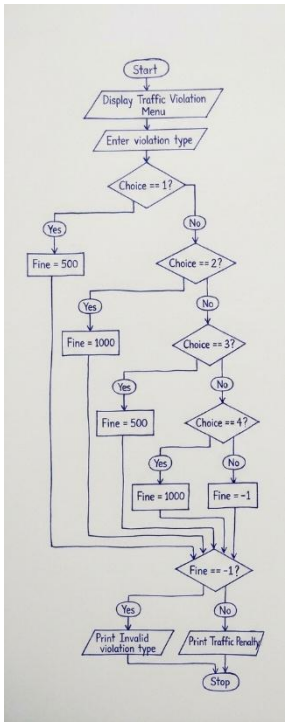
4. Algorithm

Algorithm: Traffic Violation Penalty Calculator

1. Start.

2. Display the traffic violation menu:
    - No Helmet
    - Overspeeding
    - No Seat Belt
    - Signal Violation
3. Read the violation type from the user.
4. Check the selected choice:
    - If choice is 1, assign fine = 500.
    - If choice is 2, assign fine = 1000.
    - If choice is 3, assign fine = 500.
    - If choice is 4, assign fine = 1000.
    - Otherwise, assign fine = -1.
5. Check whether the fine is equal to -1.
6. If the fine is -1, display "Invalid violation type".
7. Otherwise, display the traffic penalty amount.
8. Stop.

5. Flowchart



6. Pseudocode

BEGIN

PRINT "1. No Helmet"

PRINT "2. Overspeeding"

PRINT "3. No Seat Belt"

PRINT "4. Signal Violation"

INPUT choice

IF choice = 1 THEN

```
    fine = 500

ELSE IF choice = 2 THEN

    fine = 1000

ELSE IF choice = 3 THEN

    fine = 500

ELSE IF choice = 4 THEN

    fine = 1000

ELSE

    fine = -1

END IF

IF fine = -1 THEN

    PRINT "Invalid violation type"

ELSE
```

PRINT "Traffic Penalty: Rs.", fine

END IF

END

8. Python Source Code

```
print("1. No Helmet")
print("2. Overspeeding")
print("3. No Seat Belt")
print("4. Signal Violation")

choice = int(input("Enter violation type: "))

if choice == 1:
    fine = 500
elif choice == 2:
    fine = 1000
elif choice == 3:
    fine = 500
elif choice == 4:
    fine = 1000
else:
    fine = -1

if fine == -1:
    print("Invalid violation type")
else:
    print("Traffic Penalty: Rs.", fine)
```

8. Test Cases

The following test cases are used to verify the correctness of the program.

Test Case	Input Choice	Violation Type	Expected Output
TC1	1	No Helmet	Traffic Penalty: Rs. 500
TC2	2	Overspeeding	Traffic Penalty: Rs. 1000

TC3	3	No Seat Belt	Traffic Penalty: Rs. 500

Test Case Calculations

TC1: No Helmet

Input:

Choice=1

Fine assigned:

Fine=500

Result: Traffic Penalty: Rs. 500.

TC2: Overspeeding

Input:

Choice=2

Fine assigned:

Fine=1000

Result: Traffic Penalty: Rs. 1000.

TC3: No Seat Belt

Input:

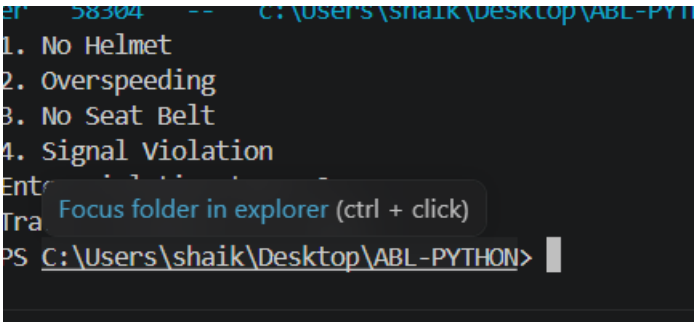
Choice=3

Fine assigned:

Fine=500

Result: Traffic Penalty: Rs. 500.

9.Output Screenshots



10. Conclusion

The Traffic Violation Penalty Calculator program was successfully developed using Python programming concepts.

The program displays different types of traffic violations and assigns the appropriate penalty amount based on the user's selection. It also validates the input and displays an error message for invalid violation types.

This case study helped in understanding input/output statements, variables, conditional statements, and if-elif-else decision-making structures in Python.

8. Laboratory Chemical Inventory Validation

1. Problem Statement

Laboratories require proper management of chemical inventories to ensure that sufficient chemicals are available for conducting experiments. Before performing an experiment, it is necessary to verify whether the available chemical quantity meets the required quantity.

The objective of this case study is to develop a Python program that validates the available chemical stock and determines whether an experiment can be conducted.

The program accepts the available chemical quantity, required quantity, and minimum stock level in millilitres. It validates the input quantities and checks whether sufficient chemical is available.

If the available quantity is sufficient, the program calculates the remaining chemical quantity and checks whether the stock is low. If the available quantity is insufficient, it calculates and displays the additional quantity required.

2. Objectives

- To understand input and output operations in Python.
- To validate chemical quantities using conditional statements.
- To calculate the remaining chemical quantity.
- To determine whether sufficient chemical is available.
- To identify low chemical stock using a minimum stock level.
- To calculate the additional chemical quantity required.
- To develop a simple laboratory chemical inventory validation system.

3. Assumptions

1. All chemical quantities are entered in millilitres (ml).
2. Available quantity, required quantity, and minimum stock level must be greater than or equal to zero.
3. The experiment can be conducted only if the available quantity is greater than or equal to the required quantity.
4. The remaining quantity is calculated after conducting the experiment.
5. Chemical stock is considered low when the remaining quantity is less than the minimum stock level.
6. If the remaining quantity is equal to the minimum stock level, the stock is considered sufficient.
7. The program assumes that the user enters valid numerical values.

4. Algorithm

Algorithm: Laboratory Chemical Inventory Validation

1. Start.

2. Read the available chemical quantity.
3. Read the required chemical quantity.
4. Read the minimum stock level.
5. Check whether any quantity is less than zero.
6. If any quantity is negative, display "Invalid quantity".
7. Otherwise, check whether the available quantity is greater than or equal to the required quantity.
8. If sufficient chemical is available:
    - Calculate the remaining quantity:

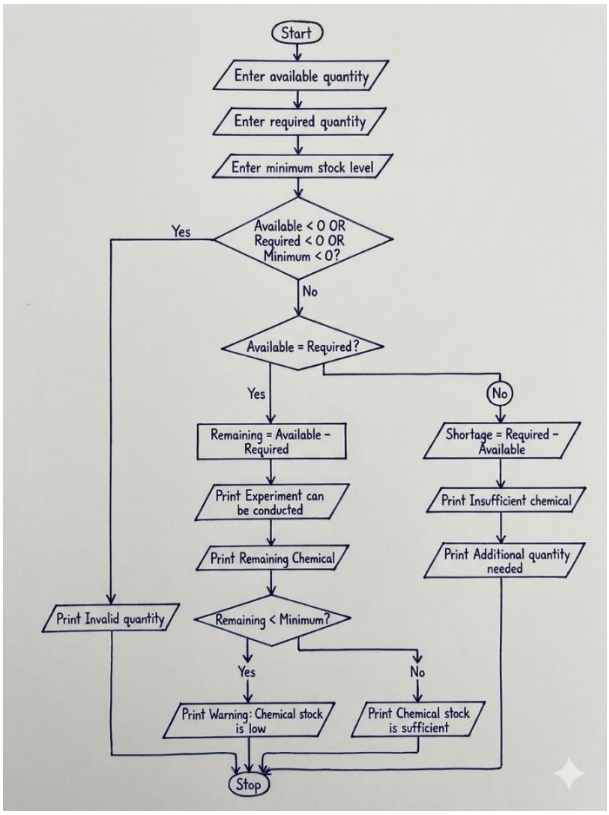
- Display "Experiment can be conducted".
- Display the remaining chemical quantity.
- Check whether the remaining quantity is less than the minimum stock level.
- If true, display "Warning: Chemical stock is low".
- Otherwise, display "Chemical stock is sufficient".

9. If insufficient chemical is available:

- Calculate the shortage:
- Display "Insufficient chemical".
- Display the additional quantity needed.

10. Stop.

5. Flowchart



6. Pseudocode

BEGIN

INPUT available

INPUT required

INPUT minimum

IF available < 0 OR required < 0 OR minimum < 0 THEN

PRINT "Invalid quantity"

ELSE

```
IF available >= required THEN
   remaining = available - required
   PRINT "Experiment can be conducted"
   PRINT "Remaining Chemical:", remaining
   IF remaining < minimum THEN
      PRINT "Warning: Chemical stock is low"
   ELSE
      PRINT "Chemical stock is sufficient"
   END IF
ELSE
   shortage = required - available
   PRINT "Insufficient chemical"
   PRINT "Additional quantity needed:", shortage
```

END IF

END IF

END

7.Python Source Code

```
available = float(input("Enter available chemical quantity (ml): "))
required = float(input("Enter required quantity (ml): "))
minimum = float(input("Enter minimum stock level (ml): "))

if available < 0 or required < 0 or minimum < 0:
    print("Invalid quantity")
else:
    if available >= required:
        remaining = available - required
        print("Experiment can be conducted")
        print("Remaining Chemical:", remaining, "ml")

        if remaining < minimum:
            print("Warning: Chemical stock is low")
        else:
            print("Chemical stock is sufficient")
    else:
        shortage = required - available
        print("Insufficient chemical")
        print("Additional quantity needed:", shortage, "ml")
```

8. Test Cases

The following test cases are used to verify the correctness of the program.

Test Case	Available (ml)	Required (ml)	Minimum (ml)	Expected Result
TC1	100	40	30	Chemical stock is sufficient

TC2	100	80	30	Warning: Chemical stock is low
TC3	50	70	20	Insufficient chemical

Test Case Calculations

TC1: Chemical Stock is Sufficient

Input:

- Available quantity = 100 ml
- Required quantity = 40 ml
- Minimum stock level = 30 ml

Remaining quantity:

100-40=60

Since 60 ml is greater than the minimum stock level of 30 ml, the chemical stock is sufficient.

TC2: Chemical Stock is Low

Input:

- Available quantity = 100 ml
- Required quantity = 80 ml
- Minimum stock level = 30 ml

Remaining quantity:

100-80=20

Since 20 ml is less than 30 ml, the program displays a low stock warning.

TC3: Insufficient Chemical

Input:

- Available quantity = 50 ml
- Required quantity = 70 ml

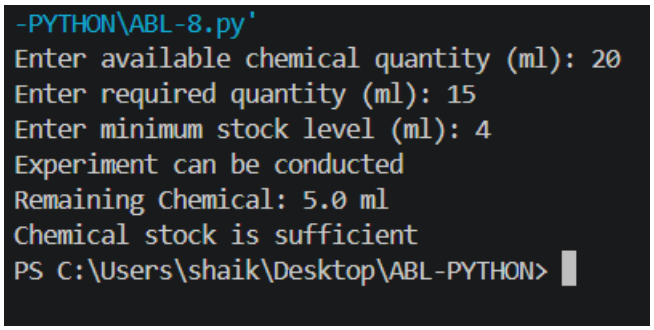
- Minimum stock level = 20 ml

Shortage:

70-50=20

Result: Additional quantity needed = 20 ml.

9. Output Screenshots



.

10. Conclusion

The Laboratory Chemical Inventory Validation program was successfully developed using Python programming concepts.

The program validates chemical quantities and determines whether sufficient chemical is available for conducting an experiment. It calculates the remaining chemical quantity, checks whether the stock is below the minimum stock level, and identifies any shortage.

This case study helped in understanding variables, input/output operations, arithmetic operators, nested conditional statements, and decision-making in Python.

The program provides a simple and effective solution for laboratory chemical inventory management and helps maintain adequate chemical stock for experiments.

9. Student Timetable Conflict Detection

1. Problem Statement

A student may have two classes scheduled on the same day, and their timings may overlap. This can cause timetable conflicts and make it difficult to attend both classes.

The objective is to develop a Python program that accepts the day, start time, and end time of two classes. It checks whether the time values are valid and detects if the classes overlap on the same day.

2. Objectives

- To accept the day and timing details of two classes.
- To validate the class timings.
- To check whether both classes occur on the same day.
- To detect overlapping class schedules.
- To display a suitable message based on the result.

3. Assumptions

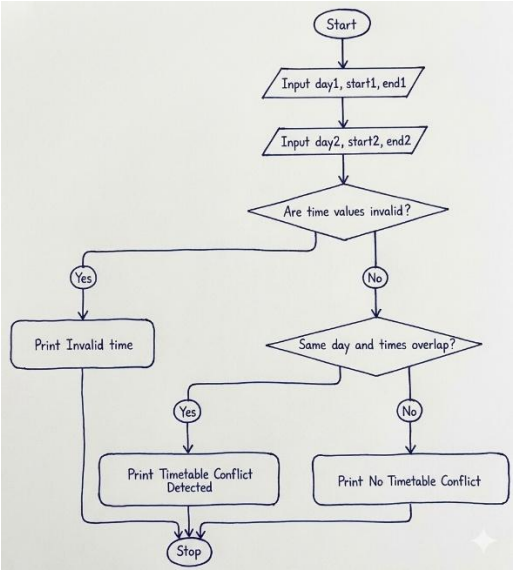
- Time is entered in minutes from midnight.
- One day contains 1440 minutes.
- Start time must be less than end time.
- Both classes are considered to conflict only if they occur on the same day and their timings overlap.
- Day names are converted to lowercase before comparison.
- The program checks only two classes at a time.

4. Algorithm

1. Start the program.
2. Read the day, start time, and end time of the first class.
3. Read the day, start time, and end time of the second class.
4. Check whether any time is negative, exceeds 1440, or has a start time greater than or equal to its end time.
5. If any invalid time is found, display "Invalid time".
6. Otherwise, check whether both classes occur on the same day and their time intervals overlap.

7. If they overlap, display "Timetable Conflict Detected".
8. Otherwise, display "No Timetable Conflict".
9. Stop the program.

5. Flowchart



6. Pseudocode

START

INPUT day1, start1, end1

INPUT day2, start2, end2

IF start1 < 0 OR end1 > 1440 OR

start2 < 0 OR end2 > 1440 OR

start1 >= end1 OR start2 >= end2 THEN

PRINT "Invalid time"

ELSE IF day1 == day2 AND

start1 < end2 AND start2 < end1 THEN

PRINT "Timetable Conflict Detected"

ELSE

PRINT "No Timetable Conflict"

END IF

STOP

28. Python Source Code

```
#Student Timetable Conflict Detection
day1 = input("Enter day of first class: ").lower()
start1 = int(input("Enter first class start time (minutes): "))
end1 = int(input("Enter first class end time (minutes): "))

day2 = input("Enter day of second class: ").lower()
start2 = int(input("Enter second class start time (minutes): "))
end2 = int(input("Enter second class end time (minutes): "))

if (start1 < 0 or end1 > 1440 or start2 < 0 or end2 > 1440
        or start1 >= end1 or start2 >= end2):
    print("Invalid time")
elif day1 == day2 and start1 < end2 and start2 < end1:
    print("Timetable Conflict Detected")
else:
    print("No Timetable Conflict")

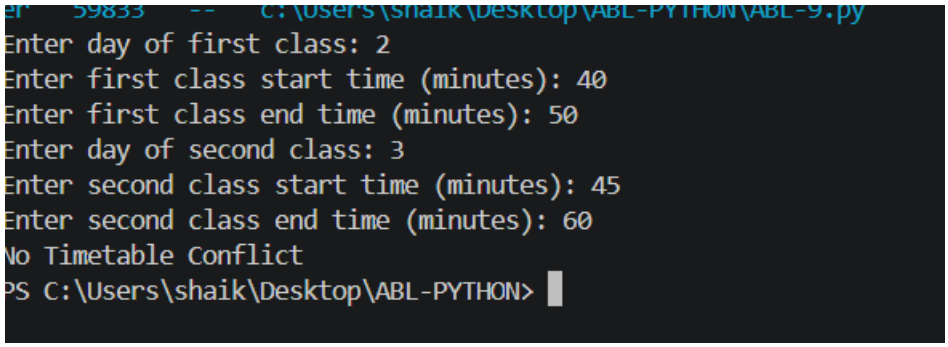
```

8. Test Cases

Test Case	First Class	Second Class	Expected Output
1	Monday, 540–600	Monday, 570–630	Timetable Conflict Detected
2	Monday, 540–600	Tuesday, 570–630	No Timetable Conflict
3	Monday, 540–600	Monday, 600–660	No Timetable Conflict
4	Monday, 540–600	Monday, 610–660	No Timetable Conflict
5	Monday, -10–600	Monday, 610–660	Invalid time

Note: The time values are in minutes from midnight. For example, 540 minutes = 9:00 AM, and 600 minutes = 10:00 AM.

9. Sample Output Screenshots



.

10. Conclusion

The Student Timetable Conflict Detection program checks the validity of class timings and identifies overlapping schedules. It helps students detect timetable conflicts and plan their classes more effectively. The program uses conditional statements and logical operators in Python to produce the correct result.

10. Courier Charge Optimization

1. Problem Statement

Different courier services charge different amounts for delivering parcels. The charges may include a fixed base fee and an additional cost based on the parcel's weight.

The objective is to develop a Python program that calculates the delivery charges of two courier services based on the parcel weight. The program compares the charges, identifies the cheaper courier, and calculates the amount saved. If both courier services charge the same amount, the program displays that their charges are equal.

2. Objectives

- To accept parcel weight from the user.
- To validate the parcel weight.
- To calculate the charges of two courier services.
- To compare the delivery charges.
- To identify the cheaper courier service.
- To calculate the amount saved by choosing the cheaper service.

3. Assumptions

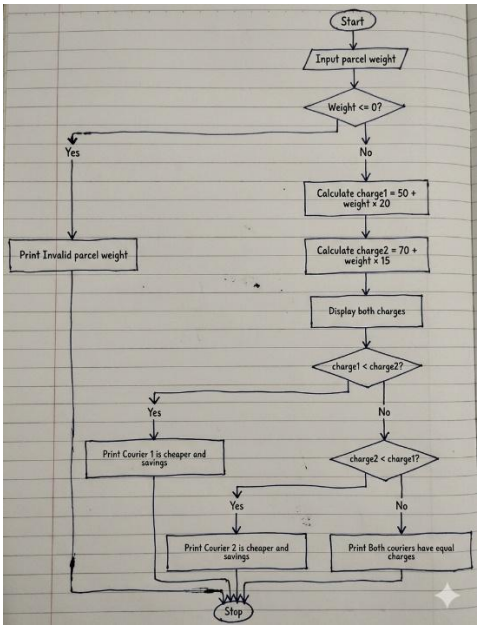
- Parcel weight is entered in kilograms.
- Parcel weight must be greater than zero.
- Courier 1 charges Rs. 50 as a base fee and Rs. 20 per kg.
- Courier 2 charges Rs. 70 as a base fee and Rs. 15 per kg.
- Charges are rounded to two decimal places for display.
- The program compares only these two courier services.

4. Algorithm

1. Start the program.
2. Read the parcel weight in kilograms.

3. Check whether the weight is less than or equal to zero.
4. If the weight is invalid, display "Invalid parcel weight".
5. Otherwise, calculate Courier 1 charge using:
    - Base charge = Rs. 50
    - Per kg charge = Rs. 20
6. Calculate Courier 2 charge using:
    - Base charge = Rs. 70
    - Per kg charge = Rs. 15
7. Display both courier charges.
8. Compare the two charges:
    - If Courier 1 is cheaper, display Courier 1 and the savings.
    - If Courier 2 is cheaper, display Courier 2 and the savings.
    - If both charges are equal, display an equal-charge message.
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START

INPUT weight

IF weight <= 0 THEN
   PRINT "Invalid parcel weight"

ELSE
   charge1 = 50 + weight * 20
   charge2 = 70 + weight * 15

   PRINT charge1
   PRINT charge2

   IF charge1 < charge2 THEN
      PRINT "Courier 1 is cheaper"
      PRINT charge2 - charge1

   ELSE IF charge2 < charge1 THEN
      PRINT "Courier 2 is cheaper"
      PRINT charge1 - charge2

   ELSE
      PRINT "Both couriers have equal charges"
```

END IF

END IF

STOP

32. Python Source Code

```
#Courier Charge Optimization
weight = float(input("Enter parcel weight in kg: "))

if weight <= 0:
    print("Invalid parcel weight")
else:
    # Courier 1: Rs. 50 base + Rs. 20 per kg
    charge1 = 50 + weight * 20

    # Courier 2: Rs. 70 base + Rs. 15 per kg
    charge2 = 70 + weight * 15

    print("Courier 1 Charge: Rs.", round(charge1, 2))
    print("Courier 2 Charge: Rs.", round(charge2, 2))

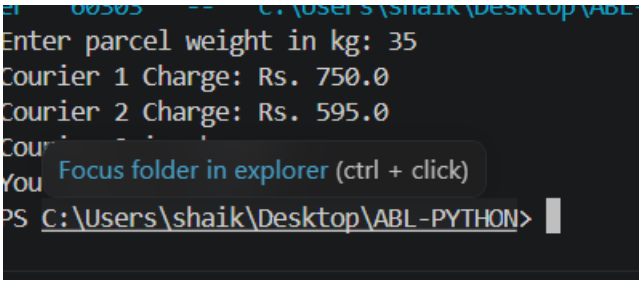
    if charge1 < charge2:
        print("Courier 1 is cheaper")
        print("You save Rs.", round(charge2 - charge1, 2))
    elif charge2 < charge1:
        print("Courier 2 is cheaper")
        print("You save Rs.", round(charge1 - charge2, 2))
    else:
        print("Both couriers have equal charges")
```

8. Test Cases

Test Case	Parcel Weight	Expected Result
1	2 kg	Courier 1 is cheaper
2	5 kg	Courier 2 is cheaper
3	4 kg	Both couriers have equal charges
4	0 kg	Invalid parcel weight

5	-3 kg	Invalid parcel weight

9.Sample Output Screenshots

10. 

10. Conclusion

The Courier Charge Optimization program calculates and compares the delivery charges of two courier services based on parcel weight. It identifies the cheaper service and displays the amount saved. This program demonstrates the use of arithmetic operations, conditional statements, and input validation in Python.

-